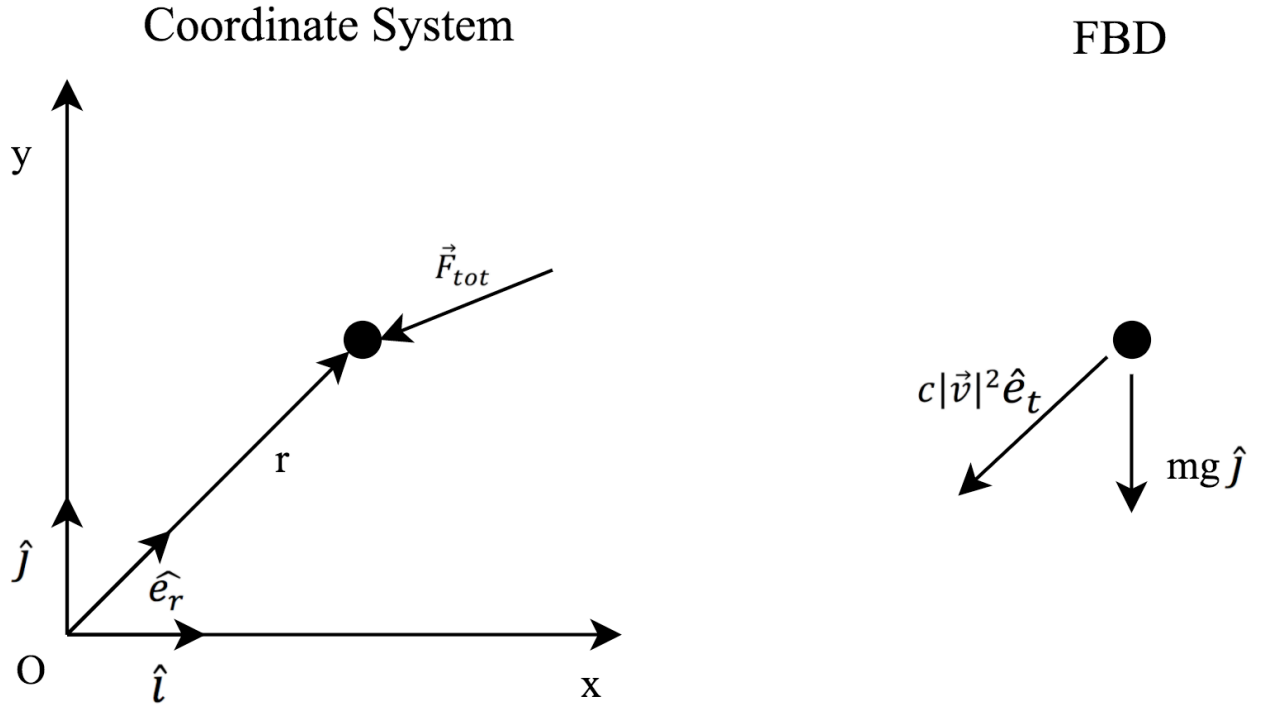


Problem 8: Cannonball 2

Christopher LaBruna

Approved by Andy Ruina, 10/11/2017

(a) Find a numerical solution using $\theta = \frac{\pi}{4}$, $v_o = 1$ m/s, $m = 1$ kg.



$$\mathbf{LMB}_{/O} \Rightarrow \Sigma \vec{F} = m\ddot{\vec{r}} = -mg\hat{j} - c|\vec{v}|^2\hat{e}_t$$

Where m is the mass of the projectile, g is the acceleration due to gravity, c is the drag coefficient, and $\vec{v}$ is the velocity. We can write the unit vector $\hat{e}_t$, tangent to the path, as $\frac{\vec{v}}{|\vec{v}|}$. Updating the $\mathbf{LMB}_{/O}$ gives:

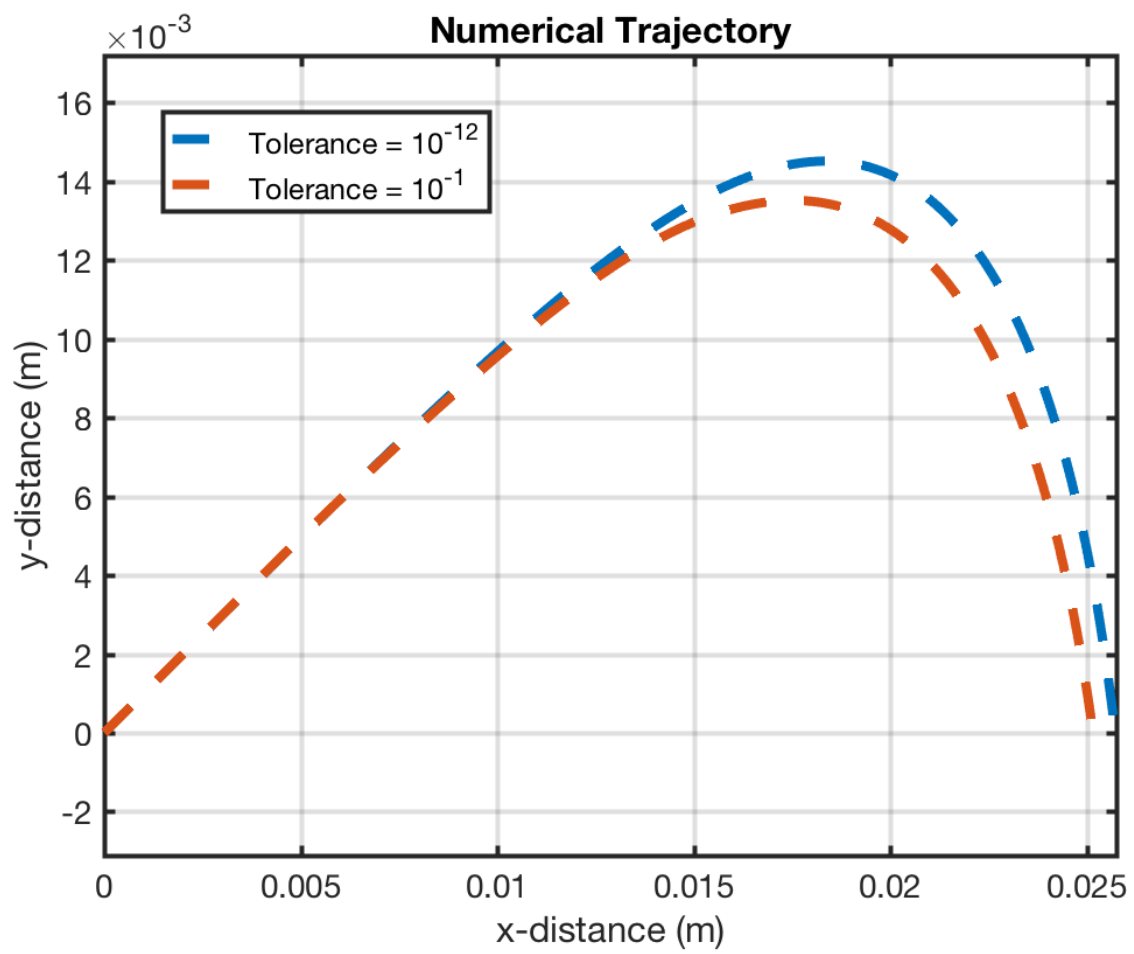
$$m\ddot{\vec{r}} = -mg\hat{j} - \frac{|\vec{v}|^2\vec{v}}{|\vec{v}|}$$

$$m\ddot{\vec{r}} = -mg\hat{j} - |\vec{v}|\vec{v}$$

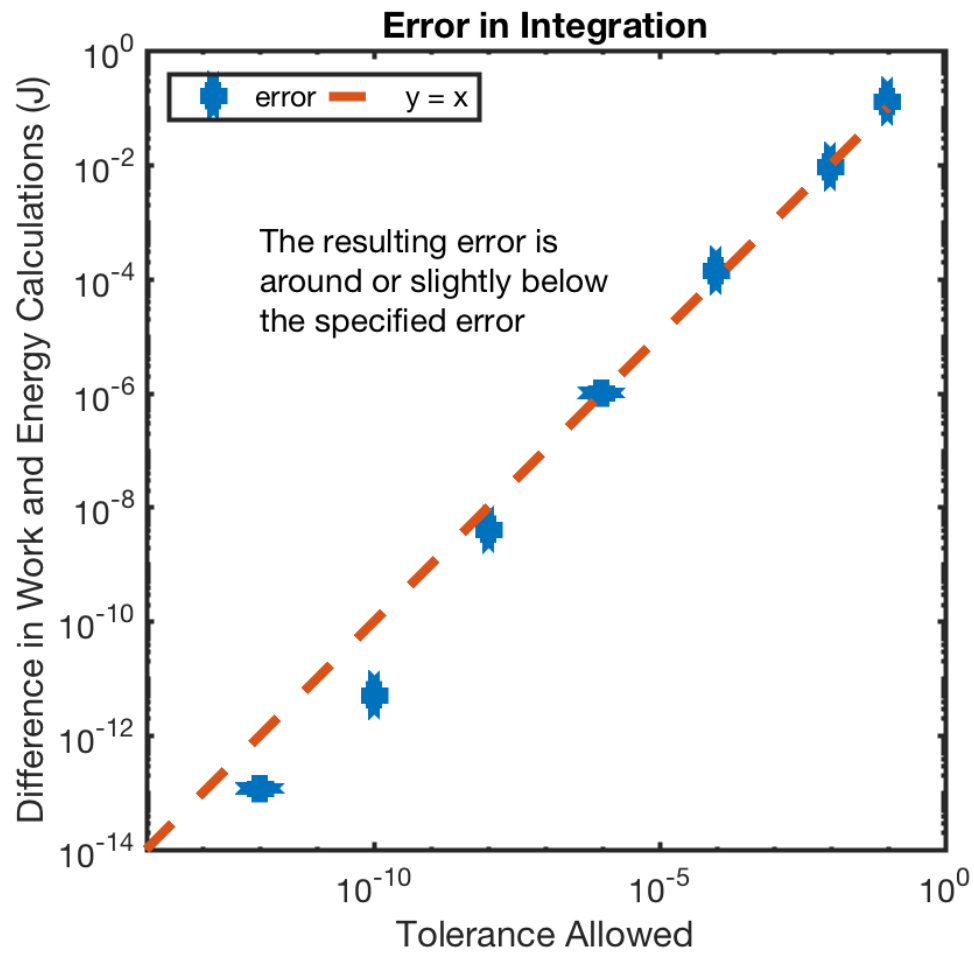
Dotting the $\mathbf{LMB}_{/O}$ with $\hat{i}$ and $\hat{j}$ gives:

$m\ddot{x} = -c \vec{v} \dot{x}$ $m\ddot{y} = -mg - c \vec{v} \dot{y}$
--

Plug into ode45 in MATLAB and obtain the following plot (for two different error tolerances):



(b) Show the integrated power calculated with numerical integration approaches the change of total energy as the integration tolerance approaches zero.



Matlab Code

```
1 function Question_8()
2 close all; clc;
3 %% Properties and initial conditions
4
5 % Properties
6 p.g = 1; % gravity constant (m/(s^2))
7 p.m = 1; % mass property
8 p.c = 100; % damping property
9 tol = [1E-1,1E-2,1E-4,...
10 1E-6,1E-8,1E-10,1E-12]; % define integration step sizes
11 x = logspace(-14,-1); % create x vector for plotting
12 y = x; % creat y vector for plotting
13 error = zeros(1,length(tol)); % create vector to store error
14 ynum = 1; % initialize ynum
15
16 % Initial conditions
17 theta = 45; % launch angle in degrees
18 speed = 1; % initial speed in (m/s)
19 v0 = speed*[cosd(theta); sind(theta)]; % initial velocity [v0x; v0y]
20 r0 = [0; 0]; % initial position [x0; y0]
21 z0 = [r0; v0;0]; % initial state vector
22 should_plot = 0; % should the trajector be plotted
23
24 %% to find work comparison graph, put everything in a loop
25 for n = 1:length(tol)
26
27
28     %% Stuff for the solver
29     tspan = 0:0.003:.5; % define time span
30     options = odeset('abstol',tol(n),'reltol',tol(n)); % define error tol
31
32     %% Solver call statement
33     [~, zarray] = ode45(@projectile, tspan, z0, options, p);
34
35     if n == 1
36         xnum_bad = zarray(:,1); % x position
37         ynum_bad = zarray(:,2); % y position
38
39     elseif n == length(tol)
```

```

40     should_plot = 1; % the trajectory should be plotted
41     ynum_bad_short = ynum_bad(1:last_index-4);
42     xnum_bad_short = xnum_bad(1:last_index-4);
43 end
44
45 % Unpack these values for easier plotting
46 xnum = zarray(:,1); % x position
47 ynum = zarray(:,2); % y position
48 vx = zarray(:,3); % x velocity
49 vy = zarray(:,4); % y velocity
50 W = zarray(:,5); % get work
51 m = p.m; % unpack mass
52 g = p.g; % unpack gravity
53 len = length(zarray) - 1; % calc length for loops
54 should_continue = 1;
55 idx = 1; % initialize looping index
56
57 while should_continue == 1
58     if ynum(idx) < 0
59         last_index = idx;
60         should_continue = 0;
61         idx = idx+1;
62     end
63     if idx == length(ynum)
64         should_continue = 0;
65     end
66     idx = idx + 1; % loop through ynum
67 end
68
69 xnum_short = xnum(1:last_index); % create x subvector
70 ynum_short = ynum(1:last_index); % create y subvector
71
72 %% Find change in total energy
73 v1 = sqrt(vx(1)^2 + vy(1)^2); %find initial velocity
74 ke1 = 0.5*m*v1^2; %find initial KE
75 v2 = sqrt(vx(end)^2 + vy(end)^2); %final velocity
76 ke2 = 0.5*m*v2^2; %find final KE
77 if ynum(end) <= 0
78     u1 = m*g*(ynum(1) - ynum(end)); %find initial potential energy
79     u2 = 0; %find final potential energy

```

```

80     else
81         u1 = 0; %find initial potential energy
82         u2 = m*g*(ynum(end) - ynum(1)); %find final potential energy
83     end
84     delta_E = (ke2 +u2)-(ke1 +u1); %find change in energy
85     error(n) = abs(W(end) - delta_E); %find diff. in work and energy
86
87 %% Plot trajectory
88 if should_plot == 1
89
90     figure
91     P = plot(xnum_short,ynum_short,'—'); %plot x,y trajectory
92     grid on
93     set(P,'LineWidth',4); %set linewidth of curve
94     set(gca,'Box','on','LineWidth',2,...
95         'FontName','Helvetica',...
96         'FontSize',14); %set axis properties
97     title('Numerical Trajectory')
98     xlabel('x-distance (m)')
99     ylabel('y-distance (m)')
100    axis('equal')
101    hold on
102    Q = plot(xnum_bad_short,ynum_bad_short,'—'); %plot x,y trajectory
103    set(Q,'LineWidth',4); %set linewidth of curve
104    set(gca,'Box','on','LineWidth',2,...
105        'FontName','Helvetica',...
106        'FontSize',14); %set axis properties
107    legend('tolerance = 10^{-12}','tolerance = 10^{-1}');
108 end
109
110 end
111
112 %% Plot error as a function of tolerance
113 figure
114 P = loglog(tol, error, '*'); %plot error vs. tolerance
115 set(P,'LineWidth',12); %set linewidth of curve
116 set(gca,'Box','on','LineWidth',2,...
117     'FontName','Helvetica',...
118     'FontSize',14); %set axis properties
119 title('Error in Integration')

```

```

120 ylabel('Difference in work and energy calculations (J)')
121 xlabel('tolerance allowed')
122 hold on
123 Q = loglog(x,y,'—');
124 set(Q,'LineWidth',4); %set linewidth of curve
125 set(gca,'Box','on','LineWidth',2,...
126     'FontName','Helvetica',...
127     'FontSize',14); %set axis properties
128 legend('error','y = x','Location','northwest','Orientation','horizontal');
129 axis('equal')
130 axis([1E-14,1,1E-14,1]);
131 txt = ['The resulting error is' newline 'around or slightly below' newline '
132     the specified error'];
133 text(1E-12,1E-4,txt,'FontSize',14);
134
135 %% Function returning the ODE in first order form
136 function zdot = projectile(time, state, prop)
137
138 % unpack states
139 x = state(1); %x position
140 y = state(2); %y position
141 vx = state(3); %x velocity
142 vy = state(4); %y velocity
143 %wdot = state(5); %power
144
145 % unpack constants
146 g = prop.g; %gravity
147 c = prop.c; %drag constant
148 m = prop.m; %mass
149 rdot = sqrt(vx^2+vy^2); %total velocity
150
151 % Write the ODE in first order form:
152 zdot = [vx;...
153     vy;...
154     (-c/m)*vx*rdot;...
155     -g-(c/m)*vy*rdot;...
156     (-c/m)*rdot^3];
157 end

```